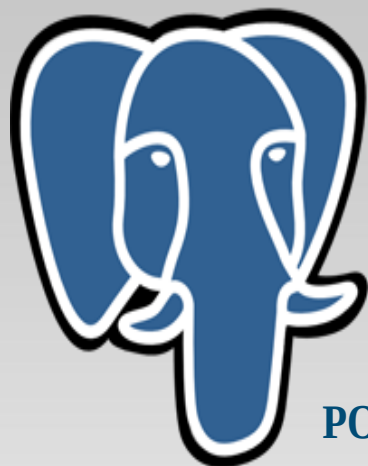




# PostgreSQL

## POSTGRES DATABASE ADMINISTRATION



STEPHEN NJOROGHE

LinkedIn: [www.linkedin.com/in/stephen-njoroge](https://www.linkedin.com/in/stephen-njoroge)

|SENIOR DATABASE ADMINISTRATOR|ORACLE| POSTGRESQL | MSSQL|OCI|AWS|

### **CHAPTER 6: Postgresql Routine Maintenance.**

#### About the Author:

I am **Stephen Njoroge**, a Senior Database Engineer with over 8+ years of expertise in designing, securing and optimizing high-performance database systems across the banking and fintech sectors. My core technical proficiency spans major databases: Oracle, PostgreSQL, MS SQL Server and MySQL-along with advanced scripting in SQL and Python for automation and data pipeline development. I architect and manage cloud and virtualization solutions on OCI, AWS, and Proxmox for onprem systems and implement comprehensive DevOps practices using monitoring tools like Prometheus and Grafana and IT service management via iTop. Further, I deliver advanced data analytics and visualization using Tableau, Power BI and Oracle BI. As a Certified Data Protection Officer, I integrate robust data protection, compliance and security measures including vulnerability assessment and role-based access control into every layer of the data infrastructure.

# Prerequisites & Setup

Create a database and tables with some data to work with.

-- Create maintenance\_practice database

```
CREATE DATABASE maintenance_practice;
```

```
postgres=#
postgres=#
postgres=#
postgres=# CREATE DATABASE maintenance_practice;
CREATE DATABASE
postgres=# |
```

-- Connect to the new database

```
\c maintenance_practice
```

```
postgres=# \c maintenance_practice
You are now connected to database "maintenance_practice" as user "postgres".
maintenance_practice=#
```

-- Create a sample table

```
CREATE TABLE employee_data (
    id SERIAL PRIMARY KEY,
    name TEXT,
    department TEXT,
    salary NUMERIC,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- Insert initial sample data

```
INSERT INTO employee_data (name, department, salary)
SELECT
    'Employee ' || generate_series,
    CASE (random() * 3)::INT
        WHEN 0 THEN 'Engineering'
        WHEN 1 THEN 'Sales'
        WHEN 2 THEN 'Marketing'
        ELSE 'HR'
    END,
    (random() * 100000 + 40000)::INT
FROM generate_series(1, 10000);
```

```

maintenance_practice=# CREATE TABLE employee_data (
maintenance_practice=#     id SERIAL PRIMARY KEY,
maintenance_practice=#     name TEXT,
maintenance_practice=#     department TEXT,
maintenance_practice=#     salary NUMERIC,
maintenance_practice=#     last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
maintenance_practice=# );
CREATE TABLE
maintenance_practice=# INSERT INTO employee_data (name, department, salary)
maintenance_practice=# SELECT
maintenance_practice=#     'Employee ' || generate_series,
maintenance_practice=#     CASE (random() * 3)::INT
maintenance_practice=#         WHEN 0 THEN 'Engineering'
maintenance_practice=#         WHEN 1 THEN 'Sales'
maintenance_practice=#         WHEN 2 THEN 'Marketing'
maintenance_practice=#         ELSE 'HR'
maintenance_practice=#     END,
maintenance_practice=#     (random() * 100000 + 40000)::INT
maintenance_practice=# FROM generate_series(1, 10000);
INSERT 0 10000
maintenance_practice=# |

```

## Using ANALYZE to Update Statistics

-- Check current statistics for our table

```

SELECT schemaname, tablename, attname, n_distinct, null_frac
FROM pg_stats
WHERE tablename = 'employee_data';

```

```

maintenance_practice=# SELECT schemaname, tablename, attname, n_distinct, null_frac
maintenance_practice=# FROM pg_stats
maintenance_practice=# WHERE tablename = 'employee_data';

```

| schemaname | tablename     | attname      | n_distinct | null_frac |
|------------|---------------|--------------|------------|-----------|
| public     | employee_data | id           | -1         | 0         |
| public     | employee_data | name         | -1         | 0         |
| public     | employee_data | department   | 4          | 0         |
| public     | employee_data | salary       | -0.9516    | 0         |
| public     | employee_data | last_updated | 1          | 0         |

```

(5 rows)

maintenance_practice=# |

```

-- Run a query and see its plan.

```

EXPLAIN ANALYZE SELECT * FROM employee_data WHERE department = 'Engineering';

```

```

maintenance_practice=# EXPLAIN ANALYZE SELECT * FROM employee_data WHERE department = 'Engineering';

```

```

QUERY PLAN
-----
Seq Scan on employee_data (cost=0.00..214.00 rows=1641 width=38) (actual time=0.067..1.383 rows=1641.00 loops=1)
  Filter: (department = 'Engineering'::text)
  Rows Removed by Filter: 8359
  Buffers: shared hit=89
Planning:
  Buffers: shared hit=34
Planning Time: 4.224 ms
Execution Time: 1.509 ms
(8 rows)

maintenance_practice=# |

```

-- Now, run ANALYZE on a table.

```
ANALYZE VERBOSE employee_data;
```

```
maintenance_practice=# ANALYZE VERBOSE employee_data;
INFO:  analyzing "public.employee_data"
INFO:  "employee_data": scanned 89 of 89 pages, containing 10000 live rows and 0 dead rows; 10000 rows in sample, 10000 estimated total rows
INFO:  finished analyzing table "maintenance_practice.public.employee_data"
avg read rate: 0.000 MB/s, avg write rate: 0.065 MB/s
buffer usage: 336 hits, 0 reads, 1 dirtied
WAL usage: 23 records, 0 full page images, 2934 bytes, 0 buffers full
system usage: CPU: user: 0.09 s, system: 0.01 s, elapsed: 0.12 s
ANALYZE
maintenance_practice=# |
```

-- Check the statistics again. They should now show real data.

```
SELECT schemaname, tablename, attname, n_distinct, null_frac
FROM pg_stats
WHERE tablename = 'employee_data';
```

```
maintenance_practice=# SELECT schemaname, tablename, attname, n_distinct, null_frac
maintenance_practice=# FROM pg_stats
maintenance_practice=# WHERE tablename = 'employee_data';
 schemaname |  tablename  |  attname  | n_distinct | null_frac
-----+-----+-----+-----+-----
 public    | employee_data | salary    |      -0.9516 |      0
 public    | employee_data | last_updated |         1 |      0
 public    | employee_data | id        |         -1 |      0
 public    | employee_data | name      |         -1 |      0
 public    | employee_data | department |          4 |      0
(5 rows)

maintenance_practice=# |
```

-- Re-run the explain plan. The estimates should be much more accurate.

```
EXPLAIN ANALYZE SELECT * FROM employee_data WHERE department = 'Engineering';
```

```
maintenance_practice=# EXPLAIN ANALYZE SELECT * FROM employee_data WHERE department = 'Engineering';
               QUERY PLAN
-----
Seq Scan on employee_data (cost=0.00..214.00 rows=1641 width=38) (actual time=0.042..1.900 rows=1641.00 loops=1)
  Filter: (department = 'Engineering'::text)
  Rows Removed by Filter: 8359
  Buffers: shared hit=89
Planning:
  Buffers: shared hit=21 dirtied=1
Planning Time: 0.467 ms
Execution Time: 2.003 ms
(8 rows)

maintenance_practice=# |
```

ANALYZE scans the table to build statistics for the query planner .

The VERBOSE option shows the progress of the operation .

Accurate statistics lead to better query plans .

## Understanding VACUUM

VACUUM reclaims storage occupied by "dead tuples" created by UPDATE and DELETE operations .  
Check the current size and live/dead tuples of the table

```
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    pg_size_pretty(pg_total_relation_size(relid)) as total_size
FROM pg_stat_user_tables
WHERE relname = 'employee_data';
```

```
maintenance_practice=# SELECT
maintenance_practice=#     relname,
maintenance_practice=#     n_live_tup,
maintenance_practice=#     n_dead_tup,
maintenance_practice=#     pg_size_pretty(pg_total_relation_size(relid)) as total_size
maintenance_practice=# FROM pg_stat_user_tables
maintenance_practice=# WHERE relname = 'employee_data';
   relname   | n_live_tup | n_dead_tup | total_size
-----+-----+-----+-----
employee_data |       10000 |           0 | 992 kB
(1 row)

maintenance_practice=# |
```

Perform a massive update .

```
UPDATE employee_data SET salary = salary * 1.1;
```

```
maintenance_practice=# UPDATE employee_data SET salary = salary * 1.1;
UPDATE 10000
maintenance_practice=# |
```

Check the statistics again. n\_dead\_tup should have increased.

```
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    pg_size_pretty(pg_total_relation_size(relid)) as total_size
FROM pg_stat_user_tables
WHERE relname = 'employee_data';
```

```

maintenance_practice=# SELECT
maintenance_practice=#         relname,
maintenance_practice=#         n_live_tup,
maintenance_practice=#         n_dead_tup,
maintenance_practice=#         pg_size_pretty(pg_total_relation_size(relid)) as total_size
maintenance_practice=# FROM pg_stat_user_tables
maintenance_practice=# WHERE relname = 'employee_data';
      relname      | n_live_tup | n_dead_tup | total_size
-----+-----+-----+-----
employee_data |      10000 |           0 | 1936 kB
(1 row)

maintenance_practice=#

```

-- Run a standard VACUUM to clean up dead tuples

```
VACUUM VERBOSE employee_data;
```

```

maintenance_practice=# VACUUM VERBOSE employee_data;
INFO:  vacuuming "maintenance_practice.public.employee_data"
INFO:  finished vacuuming "maintenance_practice.public.employee_data": index scans: 0
pages: 0 removed, 180 remain, 1 scanned (0.56% of total), 0 eagerly scanned
tuples: 0 removed, 10000 remain, 0 are dead but not yet removable
removable cutoff: 1910, which was 0 XIDs old when operation ended
frozen: 0 pages from table (0.00% of total) had 0 tuples frozen
visibility map: 0 pages set all-visible, 0 pages set all-frozen (0 were all-visible)
index scan not needed: 0 pages from table (0.00% of total) had 0 dead item identifiers removed
avg read rate: 0.000 MB/s, avg write rate: 0.000 MB/s
buffer usage: 12 hits, 0 reads, 0 dirtied
WAL usage: 0 records, 0 full page images, 0 bytes, 0 buffers full
system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
INFO:  vacuuming "maintenance_practice.pg_toast.pg_toast_44401"
INFO:  finished vacuuming "maintenance_practice.pg_toast.pg_toast_44401": index scans: 0
pages: 0 removed, 0 remain, 0 scanned (100.00% of total), 0 eagerly scanned
tuples: 0 removed, 0 remain, 0 are dead but not yet removable
removable cutoff: 1910, which was 0 XIDs old when operation ended
new relfrozenxid: 1910, which is 7 XIDs ahead of previous value
frozen: 0 pages from table (100.00% of total) had 0 tuples frozen
visibility map: 0 pages set all-visible, 0 pages set all-frozen (0 were all-visible)
index scan not needed: 0 pages from table (100.00% of total) had 0 dead item identifiers removed
avg read rate: 1.000 MB/s, avg write rate: 1.000 MB/s
buffer usage: 12 hits, 1 reads, 1 dirtied
WAL usage: 1 records, 1 full page images, 1835 bytes, 0 buffers full
system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
VACUUM
maintenance_practice=#

```

-- Check the statistics. n\_dead\_tup should now be 0 or very low.

-- Note: The total table size might not have shrunk, but the space is now available for reuse.

```

SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    pg_size_pretty(pg_total_relation_size(relid)) as total_size
FROM pg_stat_user_tables
WHERE relname = 'employee_data';

```

```

maintenance_practice=# SELECT
maintenance_practice=#     relname,
maintenance_practice=#     n_live_tup,
maintenance_practice=#     n_dead_tup,
maintenance_practice=#     pg_size_pretty(pg_total_relation_size(relid)) as total_size
maintenance_practice=# FROM pg_stat_user_tables
maintenance_practice=# WHERE relname = 'employee_data';
      relname      | n_live_tup | n_dead_tup | total_size
-----+-----+-----+-----
employee_data |    10000   |          0 | 1936 kB
(1 row)

maintenance_practice=# |

```

UPDATE and DELETE do not physically remove data; they mark rows as "dead" .

Standard VACUUM marks the space from dead tuples as reusable but does not usually return it to the operating system .

## VACUUM FULL for Space Reclamation

Compare standard VACUUM with VACUUM FULL and observe how FULL physically shrinks a table .

-- Get the exact size of the table before a major delete

```
SELECT pg_size_pretty(pg_total_relation_size('employee_data')) as size_before_full;
```

```

maintenance_practice=# SELECT pg_size_pretty(pg_total_relation_size('employee_data')) as size_before_full;
size_before_full
-----
2136 kB
(1 row)

maintenance_practice=# |

```

-- Delete a large portion of the data

```
DELETE FROM employee_data WHERE department = 'HR';
```

-- Run a standard VACUUM

```
VACUUM VERBOSE employee_data;
```

```

maintenance_practice=# DELETE FROM employee_data WHERE department = 'HR';
DELETE 1648
maintenance_practice=# VACUUM VERBOSE employee_data;
INFO:  vacuuming "maintenance_practice.public.employee_data"
INFO:  finished vacuuming "maintenance_practice.public.employee_data": index scans: 1
pages: 0 removed, 180 remain, 92 scanned (51.11% of total), 0 eagerly scanned
tuples: 1648 removed, 13241 remain, 0 are dead but not yet removable
removable cutoff: 1911, which was 0 XIDs old when operation ended
frozen: 0 pages from table (0.00% of total) had 0 tuples frozen
visibility map: 92 pages set all-visible, 0 pages set all-frozen (0 were all-visible)
index scan needed: 92 pages from table (51.11% of total) had 1648 dead item identifiers removed
index "employee_data_pkey": pages: 57 in total, 0 newly deleted, 0 currently deleted, 0 reusable
avg read rate: 0.000 MB/s, avg write rate: 58.842 MB/s
buffer usage: 362 hits, 0 reads, 57 dirtied
WAL usage: 334 records, 58 full page images, 213464 bytes, 0 buffers full
system usage: CPU: user: 0.01 s, system: 0.00 s, elapsed: 0.00 s
INFO:  vacuuming "maintenance_practice.pg_toast.pg_toast_44401"
INFO:  finished vacuuming "maintenance_practice.pg_toast.pg_toast_44401": index scans: 0
pages: 0 removed, 0 remain, 0 scanned (100.00% of total), 0 eagerly scanned
tuples: 0 removed, 0 remain, 0 are dead but not yet removable
removable cutoff: 1911, which was 0 XIDs old when operation ended
new relfrozenxid: 1911, which is 1 XIDs ahead of previous value
frozen: 0 pages from table (100.00% of total) had 0 tuples frozen
visibility map: 0 pages set all-visible, 0 pages set all-frozen (0 were all-visible)
index scan not needed: 0 pages from table (100.00% of total) had 0 dead item identifiers removed
avg read rate: 0.000 MB/s, avg write rate: 0.000 MB/s
buffer usage: 11 hits, 0 reads, 0 dirtied
WAL usage: 1 records, 0 full page images, 258 bytes, 0 buffers full
system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
VACUUM
maintenance_practice=# |

```

-- Check the size.

```
SELECT pg_size_pretty(pg_total_relation_size('employee_data')) as size_after_vacuum;
```

```

maintenance_practice=# SELECT pg_size_pretty(pg_total_relation_size('employee_data')) as size_after_vacuum;
 size_after_vacuum
-----
 1936 kB
(1 row)

maintenance_practice=#

```

-- Now run VACUUM FULL

```
VACUUM FULL VERBOSE employee_data;
```

```

maintenance_practice=# VACUUM FULL VERBOSE employee_data;
INFO:  vacuuming "public.employee_data"
INFO:  "public.employee_data": found 0 removable, 8352 nonremovable row versions in 180 pages
DETAIL:  0 dead row versions cannot be removed yet.
CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.01 s.
VACUUM
maintenance_practice=#

```

-- Check the size again.

```
SELECT pg_size_pretty(pg_total_relation_size('employee_data')) as size_after_full;
```

## PostgreSQL Routine Maintenance.

```

maintenance_practice=# SELECT pg_size_pretty(pg_total_relation_size('employee_data')) as size_after_full;
size_after_full
-----
832 kB
(1 row)

maintenance_practice=# |

```

VACUUM FULL acquires an ACCESS EXCLUSIVE lock on the table, blocking all other operations (even SELECT). It should only be used during maintenance windows .

## Reindexing for Performance

Identify index bloat and rebuild indexes to improve performance and reclaim space .

-- Create an additional index to work with

```
CREATE INDEX idx_employee_salary ON employee_data(salary);
```

-- Perform operations that can bloat an index (massive updates on indexed column)

```
UPDATE employee_data SET salary = salary * 1.05;
```

```

maintenance_practice=# CREATE INDEX idx_employee_salary ON employee_data(salary);
CREATE INDEX
maintenance_practice=# UPDATE employee_data SET salary = salary * 1.05;
UPDATE 7948
maintenance_practice=#

```

-- Check index size

```

SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexname::regclass)) as size
FROM pg_indexes
WHERE tablename = 'employee_data';

```

```

maintenance_practice=# SELECT
maintenance_practice=#     indexname,
maintenance_practice=#     pg_size_pretty(pg_relation_size(indexname::regclass)) as size
maintenance_practice=# FROM pg_indexes
maintenance_practice=# WHERE tablename = 'employee_data';
    indexname      | size
-----+-----
employee_data_pkey | 392 kB
idx_employee_salary | 512 kB
(2 rows)

```

-- Rebuild the index (this locks writes but not reads)

```
REINDEX INDEX idx_employee_salary;
```

```
maintenance_practice=# REINDEX INDEX idx_employee_salary;  
REINDEX  
maintenance_practice=# REINDEX INDEX idx_employee_salary;  
REINDEX  
maintenance_practice=# |
```

-- Use CONCURRENTLY to avoid locking writes

```
REINDEX INDEX CONCURRENTLY idx_employee_salary;
```

```
maintenance_practice=# REINDEX INDEX CONCURRENTLY idx_employee_salary;  
REINDEX  
maintenance_practice=#
```

-- Rebuild all indexes on the table

```
REINDEX TABLE employee_data;
```

-- Reindex the whole database

```
-- REINDEX DATABASE maintenance_practice;
```

```
maintenance_practice=# REINDEX INDEX idx_employee_salary;  
REINDEX  
maintenance_practice=# REINDEX INDEX idx_employee_salary;  
REINDEX  
maintenance_practice=# REINDEX INDEX CONCURRENTLY idx_employee_salary;  
REINDEX  
maintenance_practice=# REINDEX TABLE employee_data;  
REINDEX  
maintenance_practice=# REINDEX TABLE employee_data;  
REINDEX  
maintenance_practice=# REINDEX DATABASE maintenance_practice;  
REINDEX  
maintenance_practice=#
```

Indexes can become bloated over time.

REINDEX CONCURRENTLY is the safest way to rebuild indexes on a production system without interrupting writes.

## Monitoring Database Size

Use catalog views and functions to find the largest objects in the database.

-- Find the largest tables in the current database

```

SELECT
    relname as table_name,
    pg_size_pretty(pg_total_relation_size(relid)) as total_size,
    pg_size_pretty(pg_relation_size(relid)) as table_size,
    pg_size_pretty(pg_total_relation_size(relid) - pg_relation_size(relid)) as index_size
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 10;

```

```

maintenance_practice=# SELECT
maintenance_practice=#     relname as table_name,
maintenance_practice=#     pg_size_pretty(pg_total_relation_size(relid)) as total_size,
maintenance_practice=#     pg_size_pretty(pg_relation_size(relid)) as table_size,
maintenance_practice=#     pg_size_pretty(pg_total_relation_size(relid) - pg_relation_size(relid)) as index_size
maintenance_practice=# FROM pg_stat_user_tables
maintenance_practice=# ORDER BY pg_total_relation_size(relid) DESC
maintenance_practice=# LIMIT 10;

```

| table_name           | total_size | table_size | index_size |
|----------------------|------------|------------|------------|
| employee_data        | 1688 kB    | 1192 kB    | 496 kB     |
| employees            | 40 kB      | 8192 bytes | 32 kB      |
| projects             | 32 kB      | 8192 bytes | 24 kB      |
| technologies         | 24 kB      | 8192 bytes | 16 kB      |
| contracts            | 24 kB      | 8192 bytes | 16 kB      |
| departments          | 24 kB      | 8192 bytes | 16 kB      |
| clients              | 24 kB      | 8192 bytes | 16 kB      |
| project_technologies | 24 kB      | 8192 bytes | 16 kB      |
| companies            | 24 kB      | 8192 bytes | 16 kB      |
| databases            | 24 kB      | 8192 bytes | 16 kB      |

(10 rows)

-- Find the largest indexes

```

SELECT
    indexrelname as index_name,
    relname as table_name,
    pg_size_pretty(pg_relation_size(indexrelid)) as index_size
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 10;

```

```

maintenance_practice=# SELECT
maintenance_practice=#     indexrelname as index_name,
maintenance_practice=#     relname as table_name,
maintenance_practice=#     pg_size_pretty(pg_relation_size(indexrelid)) as index_size
maintenance_practice=# FROM pg_stat_user_indexes
maintenance_practice=# ORDER BY pg_relation_size(indexrelid) DESC
maintenance_practice=# LIMIT 10;

```

| index_name                | table_name           | index_size |
|---------------------------|----------------------|------------|
| idx_employee_salary       | employee_data        | 264 kB     |
| employee_data_pkey        | employee_data        | 192 kB     |
| departments_pkey          | departments          | 16 kB      |
| employees_pkey            | employees            | 16 kB      |
| employees_email_key       | employees            | 16 kB      |
| technologies_pkey         | technologies         | 16 kB      |
| projects_pkey             | projects             | 16 kB      |
| project_technologies_pkey | project_technologies | 16 kB      |
| servers_pkey              | servers              | 16 kB      |
| databases_pkey            | databases            | 16 kB      |

```

(10 rows)

maintenance_practice=# |

```

-- Check database-level disk usage

```

SELECT
    datname,
    pg_size_pretty(pg_database_size(datname)) as database_size
FROM pg_database
ORDER BY pg_database_size(datname) DESC;

```

```

maintenance_practice=# SELECT
maintenance_practice=#     datname,
maintenance_practice=#     pg_size_pretty(pg_database_size(datname)) as database_size
maintenance_practice=# FROM pg_database
maintenance_practice=# ORDER BY pg_database_size(datname) DESC;

```

| datname              | database_size |
|----------------------|---------------|
| salesdb              | 196 MB        |
| new_salesdb          | 157 MB        |
| pig_farm_management  | 11 MB         |
| maintenance_practice | 10 MB         |
| template1            | 8070 kB       |
| publisher_db         | 8057 kB       |
| subscriber_db        | 8001 kB       |
| testdb               | 7913 kB       |
| template0            | 7857 kB       |
| postgres             | 7246 kB       |

```

(10 rows)

```

## Identifying Unused Indexes

Find indexes that are not being used by the query planner, which are candidates for removal .

-- Reset statistics to get a clean slate (requires superuser)

```
SELECT pg_stat_reset();
```

```
maintenance_practice=# SELECT pg_stat_reset();
pg_stat_reset
-----
(1 row)

maintenance_practice=#
```

-- Run some queries to simulate workload

```
SELECT * FROM employee_data WHERE salary > 90000; -- Uses the salary index
SELECT * FROM employee_data WHERE id = 500;      -- Uses the primary key
```

-- Check index usage statistics

```
SELECT
    schemaname,
    relname AS table_name,
    indexrelname AS index_name,
    idx_scan AS number_of_scans,
    idx_tup_read AS tuples_read,
    idx_tup_fetch AS tuples_fetched
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC, relname;
```

```
maintenance_practice=# SELECT
maintenance_practice=#     schemaname,
maintenance_practice=#     relname AS table_name,
maintenance_practice=#     indexrelname AS index_name,
maintenance_practice=#     idx_scan AS number_of_scans,
maintenance_practice=#     idx_tup_read AS tuples_read,
maintenance_practice=#     idx_tup_fetch AS tuples_fetched
maintenance_practice=# FROM pg_stat_user_indexes
maintenance_practice=# ORDER BY idx_scan ASC, relname;
schemaname | table_name | index_name | number_of_scans | tuples_read | tuples_fetched
-----+-----+-----+-----+-----+-----
public | clients | clients_pkey | 0 | 0 | 0
public | companies | companies_pkey | 0 | 0 | 0
public | contracts | contracts_pkey | 0 | 0 | 0
public | databases | databases_pkey | 0 | 0 | 0
public | departments | departments_pkey | 0 | 0 | 0
public | employee_data | employee_data_pkey | 0 | 0 | 0
public | employee_data | idx_employee_salary | 0 | 0 | 0
public | employees | employees_email_key | 0 | 0 | 0
public | employees | employees_pkey | 0 | 0 | 0
public | project_technologies | project_technologies_pkey | 0 | 0 | 0
public | projects | projects_pkey | 0 | 0 | 0
public | servers | servers_pkey | 0 | 0 | 0
public | technologies | technologies_pkey | 0 | 0 | 0
(13 rows)
```

-- Find indexes with zero scans .

```

SELECT
    indexrelname as useless_index,
    tablename,
    idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
    AND indexrelname NOT LIKE '%pkey%'; -- Exclude primary key indexes

```

```

maintenance_practice=# SELECT
maintenance_practice-#     indexrelname AS useless_index,
maintenance_practice-#     relname AS table_name,
maintenance_practice-#     idx_scan
maintenance_practice-# FROM pg_stat_user_indexes
maintenance_practice-# WHERE idx_scan = 0
maintenance_practice-#     AND indexrelname NOT LIKE '%pkey%' -- Exclude primary key indexes
maintenance_practice-# ORDER BY relname;
   useless_index | table_name | idx_scan
-----+-----+-----
idx_employee_salary | employee_data |          0
employees_email_key | employees     |          0
(2 rows)

maintenance_practice=#

```